

이 페이지는 링크가 있는 모든 사용자에게 표시됩니다. ①

RAG 공동작업 어디까지??

RAG(검색 증강 생성) 시스템에서 인덱스(Index)는 방대한 문서 데이터를 컴퓨터(AI)가 순식간에 찾아볼 수 있도록 정리해 둔 '색인(찾아보기)' 또는 '검색용 데이터베이스'를 말합니다.

책 맨 뒤에 있는 '색인(찾아보기)'이나 도서관의 '도서 분류/검색 시스템'을 떠올리면 이해하기 쉽습니다.

1. 인덱스(Index)란?

100개 기업의 3종 보고서(수만 페이지의 PDF)가 있을 때, 챗봇에게 **"삼성전자 사외이사 비율 알려줘"**라고 물어보면 그 방대한 PDF를 처음부터 끝까지 다 읽고 찾을 수는 없습니다.

따라서 문서를 미리 잘게 쪼개고 컴퓨터가 검색하기 쉬운 형태(수치화된 좌표값이나 단어 키워드 사전)로 변환해 저장해 두는데, 이 검색 가능한 데이터 묶음 자체를 '인덱스'라고 부릅니다.

- **Dense 인덱스 (벡터 DB)**: 문서의 문장들을 의미 단위의 숫자 배열(벡터)로 변환해 둔 저장소 (문맥·의미 기반 검색용)
- **Sparse 인덱스 (BM25)**: 문서에 등장하는 주요 키워드와 빈도를 형태소별로 묶어둔 단어 사전 (정확한 고유명사 매칭용)

2. "인덱스가 만들어지기 전까지"가 의미하는 작업

이 표현은 "문서를 검색 엔진에 넣어서 검색 준비를 마칠 때까지의 공통 데이터 가공 과정"을 뜻합니다.

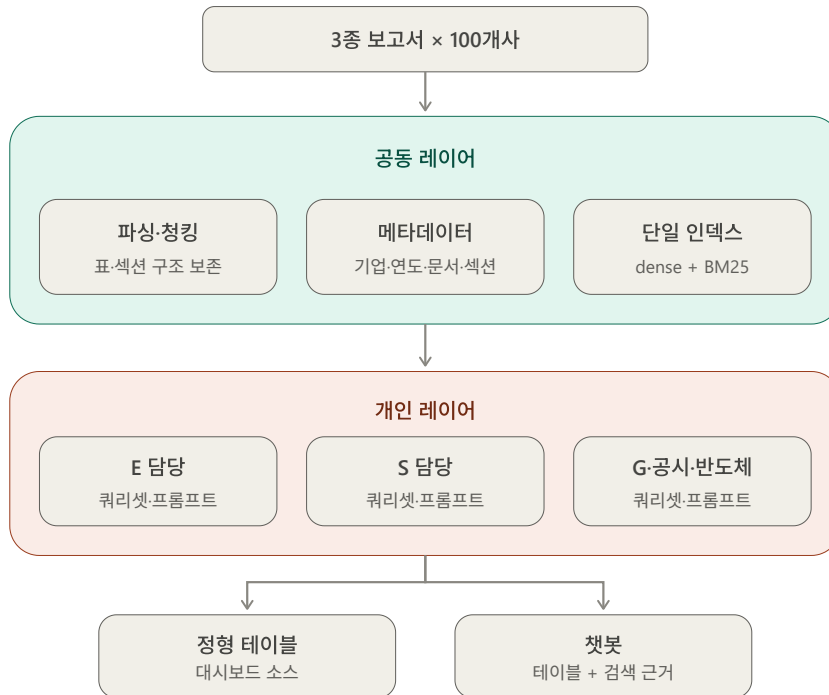
1. **문서 수집 및 파싱**: 100개사의 PDF 파일에서 텍스트와 표를 추출
2. **청킹(Chunking)**: 추출한 긴 텍스트를 적당한 크기(예: 500~800자 단위)의 토막으로 자름
3. **메타데이터 부착**: 각 토막마다 **기업명**, **연도**, **문서종류(사업/지속가능/지배구조)**, **페이지 번호** 등의 꼬리표를 달아줌
4. **임베딩 및 색인화**: 쪼갠 토막들을 임베딩 모델을 통해 벡터 DB에 넣고, BM25 키워드 사전으로 빌드

이 1~4번 과정이 완료되면 비로소 "검색 가능한 단 하나의 공통 인덱스(도서관 서가)"가 완성됩니다.

3. 왜 이게 중요할까?

- **공동 작업**: 100개사 보고서로 단 하나의 튼튼한 인덱스(검색 참고)를 다 같이 만드는 것
- **개인 작업**: 이미 만들어진 그 인덱스에서 **각자 맡은 문항(E, S, G/반도체)**에 맞는 질문을 던져서 점수를 뽑아내는 것

만약 인덱스 구축을 팀원 각자 해버리면, 똑같은 PDF를 각자 다른 방식으로 자르고 저장해서 나중에 챗봇 하나로 합칠 때 시스템이 완전히 어긋나게 됩니다.



공동작업의 범위

지난 다이어그램 기준으로 말하면, **공동작업 = 인덱스가 만들어지기 전까지의 모든 것**이야. 즉:

- PDF 파싱 → 청킹 → 메타데이터 부여 → 임베딩 → 벡터스토어 적재 → BM25 인덱스 구축
- 여기까지 나온 "하나의 검색 가능한 인덱스"가 공동 산출물

개인작업 = 그 인덱스를 갖고 각자 파트(E/S/G+공시+반도체)의 91문항을 검색·추출하는 로직이야. 쿼리 설계, 프롬프트, 채점 함수, 결과 검증.

이 경계가 중요한 이유는, 인덱스 자체를 잘못 나누면(예: 각자 PDF를 따로 파싱) 나중에 챗봇 단계에서 세 개를 다시 합쳐야 하는데 그게 사실상 재작업이기 때문이야. 반대로 개인 파트의 쿼리·프롬프트까지 회의에서 다 정하려고 하면 비효율적이야 — 그건 각자 자기 도메인 지식(G/반도체 채점기준)을 제일 잘 아는 사람이 판단할 문제니까.

회의 안건 — 팀 전체가 정해야 하는 것

인덱스에 손대는 순간 전원에게 영향을 주는 것들이라, 누군가 먼저 시작해버리면 나머지는 그 결정에 종속돼. 그래서 착수 전에 반드시 합의가 필요해.

1. 문서 파싱/청킹 전략

왜 정해야 하나: 청킹 단위와 표 처리 방식이 바뀌면 인덱스를 통째로 다시 만들어야 해. 특히 네 파트 (G-1-2 사외이사 비율, G-2-1 출석률 등)는 답이 전부 **표 안에** 있어서, 표가 깨진 채로 청킹되면 어떤 검색 전략을 쓰든 답이 안 나와.

정할 것:

- 파싱 도구 (PyMuPDF+직접 표 추출 vs unstructured.io vs LlamaParse — 표 인식 정확도로 비교)
- 청크 크기/오버랩 (예: 500~800 토큰, 오버랩 100)
- 표는 별도 청크로 분리할지, 주변 텍스트와 합칠지

2. 메타데이터 스키마

왜 정해야 하나: 이미 KCGS 등급 파일이 `company_code + year` 키로 되어 있어. 인덱스의 메타데이터가 이 키와 안 맞으면 나중에 정형 테이블과 조인이 안 돼. 스키마가 팀원마다 다르면(예: 한쪽은 "삼성 전자", 다른 쪽은 "005930") 필터링 자체가 깨져.

정할 것:

- 필수 필드 확정: `company_code`, `year`, `doc_type` (사업보고서/지배구조보고서/지속가능경영보고서), `section`, `page`, `chunk_id`
- `company_code` 포맷 통일 (종목코드)

3. 임베딩 모델

왜 정해야 하나: 모델을 바꾸면 전체 재임베딩이 필요해. 비용·속도·한국어 성능이 걸린 문제라 한 사람이 임의로 정하면 안 돼. 현재 `gemini-embedding-001` (Google AI Studio API, `GoogleGenerativeAIEmbeddings` 통해서 호출).

정할 것:

- 후보: BGE-M3(다국어+한국어 양호, dense+sparse 동시 지원), multilingual-e5-large, 업스테이지 Solar Embedding(한국어 특화, 유료 API)
- 로컬 실행 vs API 과금 — 100개사 3종 문서 규모 감안한 비용 추정 후 결정

4. 벡터스토어 및 인덱스 구조

왜 정해야 하나: 100개사를 하나의 인덱스에 넣으면, 특정 기업 질문에 다른 기업 청크가 섞여 나올 위험이 커. 이건 검색 품질에 직결돼.

정할 것:

- FAISS(가볍지만 메타필터 취약) vs Qdrant/`Chroma(company_code 필터링 네이티브 지원)`
- **단일 인덱스+메타필터**

5. BM25 토크나이저

왜 정해야 하나: 한국어는 조사 때문에 기본 공백 분리 BM25가 사실상 안 먹혀. 이거 하나가 하이브리드 서치 성능을 좌우하는데, 개인이 각자 다른 토큰라이저를 쓰면 결과가 재현이 안 돼.

정할 것:

- Kiwi vs Mecab 중 택1 (Kiwi가 설치 간편, 형태소 품질 준수)
- BM25 인덱스도 회사별 분리할지 여부 (4번 항목과 연동)

"BM25 기본 토큰라이저는 한국어 조사를 분리하지 못해 검색이 깨지므로, 팀 공통으로 설치가 가장 간편한 **Kiwi (kiwipiepy)**를 **preprocess_func** 으로 연동해서 BM25 인덱스를 만들자."

6. 하이브리드 결합 방식

왜 정해야 하나: dense와 BM25의 가중치, 리랭커 사용 여부는 도메인마다 최적값이 다를 수 있어. 기본 값을 하나 정해두고, 이후 각자 파트에서 벗어날 필요가 있으면 예외를 회의에 다시 올리는 방식이 합리적이야.

정할 것:

- EnsembleRetriever RRF 가중치 기본값 (예: dense 0.5 / BM25 0.5로 시작 후 평가셋으로 튜닝)
- 리랭커 사용 여부 및 모델 (bge-reranker-v2-m3 등 — 배치 작업이라 지연시간 부담 적음)

7. 평가셋(골드셋) 구축

왜 정해야 하나: 이게 없으면 위의 모든 튜닝(청킹 크기, 가중치, 리랭커 여부)이 감이야. 어떤 설정이 더 나은지 비교할 기준이 없어.

정할 것:

- 표본 기업 수 (5~10개사 권장) 및 어떤 기업을 고를지 (규모/업종 다양성 고려)
- 91문항 전체 정답을 만들지, 파트별 일부만 만들지
- 누가 무엇을 담당하는지 (각자 자기 파트 문항의 정답은 본인이 제일 잘 확인할 수 있음)

8. 정형 테이블 출력 스키마

왜 정해야 하나: 이 테이블이 대시보드와 챗봇 두 곳에서 공유되는 최종 산출물이야. 컬럼명, 점수 형식 (0~100 vs 단계 1~5), **need_review** 플래그 처리 방식이 파트마다 다르면 나중에 합칠 때 스키마 충돌이 나.

정할 것:

- 컬럼 구조 통일 (분류코드, 단계, 배점, confidence, 근거 청크 ID 등)
- 신뢰도 낮은 항목("Low") 처리 규칙 통일

9. 저장소/인프라 및 일정

왜 정해야 하나: 인덱스 완성 전에는 개인 작업(파트별 문항 검증)을 시작할 수가 없어. 이게 의존성 게이트라서, 완료 시점을 명확히 정해야 개인 일정도 잡혀.

정할 것:

- 코드/데이터 저장소 구조 (공동 레포 1개, 브랜치 전략)
- 인덱스 최초 버전 완료 목표일 — 이 날짜 이후에 개인 파트 착수

우선순위를 매기자면 1(청킹) → 2(메타데이터) → 4(인덱스 구조) → 5(토크나이저) → 3(임베딩) 순서로 먼저 합의해야 나머지가 진행돼. 6·7·8·9는 뼈대가 정해진 뒤 병행 가능해.

필수 선행 결정 — 반드시 순서대로



그럼 파싱 청킹 임베딩 모델은 뭐로 할것인가??

결론부터 말하면 **비용과 성능의 균형을 가장 깔끔하게 잡은 실무적인 조합**입니다.

두 조합의 특징과 실제 적용할 때 반드시 챙겨야 할 핵심 포인트를 정리해 드릴게요.

1. 파서: **pdfplumber** (무료 오픈소스)

- **선택 이유:** 텍스트뿐만 아니라 표(Table)의 경계선과 셀 데이터를 정확히 추출할 수 있어, 사업보고서와 기업지배구조보고서의 표 추출에 매우 강력합니다.
- **실전 체크포인트:**
 - 표는 Markdown 포맷으로 변환: `extract_tables()` 로 뽑은 2차원 리스트를 `| 열1 | 열2 |` 형태의 Markdown 표 문자열로 만들어 청크에 넣어야 임베딩 모델이 행·열 관계를 온전히 이해합니다.
 - 지속가능경영보고서(자유 디자인) 샘플 검증: 2~3단으로 나뉜 페이지에서 줄글 읽는 순서가 꼬이지 않는지 1~2개 기업 샘플로 텍스트 출력 순서를 꼭 눈으로 확인해 보세요.

2. 임베딩 모델 비교 및 추천

구분	BAAI/bge-m3 (로컬 오픈소스)	OpenAI text-embedding-3-small (API)
비용	0원 (완전 무료)	약 1~2달러 (커피 한 잔 값)
하이브리드 지원	Dense + Sparse를 동시에 자체 생성 (BM25를 따로 안 만들어도 됨)	Dense만 제공 (BM25는 Kiwi 등으로 별도 구축 필요)
한국어/도메인 성능	한국어 다국어 성능 최상위권, ESG 전문 용어 커버리지 우수	일반 문맥 이해도 매우 높음
인프라 요구사항	로컬 GPU(Colab T4 등) 필요	API 키만 있으면 CPU 환경에서도 초고속 실행 가능

추천 선택 기준

- **추천 1 (가장 강력한 올인원):** **BGE-M3**
 - Colab 무료 T4 GPU나 로컬 GPU가 있다면 **BGE-M3** 가 가장 유리합니다. Dense 벡터와 Sparse(키워드 가중치) 벡터를 한 번에 뽑아주기 때문에, 복잡한 BM25 형태소 분석기를 별도로 세팅하는 수고를 크게 덜 수 있습니다.
- **추천 2 (간편하고 빠른 속도):** **text-embedding-3-small**
 - GPU 환경을 세팅하기 번거롭고 빠른 속도로 파이프라인을 돌려보고 싶다면 OpenAI API를 쓰는 것이 좋습니다. 100개사를 전부 넣어도 2~3천 원 수준이라 비용 부담이 사실상 없습니다. (단, 이 경우 BM25는 **kiwipiepy** 로 따로 붙여주어야 합니다.)

3. 시작하기 전 추천 진행 순서

1. 삼성전자 + 복잡한 중소기업 1곳을 골라 `pdfplumber` 로 파싱합니다.
2. 이사회 구성 표(G 문항)와 온실가스 배출량 표(E 문항)가 **Markdown 표 형태로 깨짐 없이 변환되는지** 확인합니다.
3. 해당 데이터로 `BGE-M3` 또는 `text-embedding-3-small` 임베딩을 거쳐 검색 테스트를 3~4개 문항만 던져본 뒤, 문제없으면 100개사 전체로 확장하시면 됩니다.

최종 확정 스키마 (v1)

필드명	타입	값/기본값	결정 근거
company_code	string	"005930"	KCGS 파일 포맷과 동일, 6자리 문자열 (0 소실 방지)
company_name	string	"삼성전자"	—
year	int	2024	KCGS 파일의 float 오염 방지 위해 적재 시 int 캐스팅
doc_type	enum	business / governance / sustainability	영문 스네이크케이스로 통일
doc_id	string	"005930_2024_business"	company_code+year+doc_type 합성
chunk_id	string	"005930_2024_business_0142"	doc_id + 4자리 순번
page	int int[]	87 또는 [87,88]	페이지 경계 걸치면 배열
section_title	string null	사업/지배구조보고서: 목차 규칙 기반 추출 / 지속가능경영보고서: 일단 null, 검색 품질 낮으면 보강	문서 구조 차이 반영
content_type	enum	text / table / table_caption	표 가중치 조정용
table_data	JSON null	content_type=table 일 때 원본 셀 구조(행/열) 저장	숫자 정확도 확보, 마크다운과 병행 저장
esg_area	enum	환경 / 사회 / 지배구조 / 정보공시	채점기준표 영역 컬럼과 동일
applicable_item_codes	string[]	기본 [], 1차 인덱싱에서는 태깅 스킵	평가셋 확인 후 필요하면 추가
chunk_char_count	int	—	튜닝용
embedding_model	string	"bge-m3" (3번 항목 최종 결정값 반영)	모델 교체 추적
indexed_at	datetime	—	스테일 체크 판별
source_file_hash	string	—	중복 적재 방지

이대로 만들어서 진행하고, 회의에선 이 스키마 자체가 아니라 **1번(청킹) / 3번(임베딩 모델) / 4번(인덱스 구조) / 5번(BM25 토크나이저)**처럼 진짜로 팀원 각자의 결과물에 영향을 주고 되돌리기 비싼 것들에 시간을 쓰는 게 맞아.

평가셋 구축

[1단계: 소규모 검증 (비용 100~500원 수준)]

1. 대표 기업 5개사(삼성전자 등)만 PDF 파싱 & 청킹

2. 5개사 데이터만 임베딩 (무료 BGE-M3 or 소액 API)
3. 골드셋(표본 정답)과 AI 채점 결과 비교 및 디버깅
 - └─ 검색 실패 시: 청킹 크기 / 쿼리 / 가중치 수정 (수정 비용 0원)
 - └─ 채점 실패 시: LLM 프롬프트 수정 (수정 비용 0원)

[2단계: 전체 확장 (단 1회 실행)]

4. 골드셋 적중률이 만족스러울 때, 나머지 95개사 전체 파싱 & 임베딩
5. 100개사 전체 문항 LLM 채점 1회 완주

